Manual PyReact

README.md

? PyReact - Framework Web Declarativo para Python

[![PyPI version](https://badge.fury.io/py/pyreact-framework.svg)](https://pypi.org/project/pyreact-framework/)
[![Python](https://img.shields.io/pypi/pyversions/pyreact-framework.svg)](https://pypi.org/project/pyreact-framework/)
[![License: MIT](https://img.shields.io/badge/License-MIT-yellow.svg)](https://opensource.org/licenses/MIT)

PyReact é um framework web declarativo inspirado no React, mas construído nativamente para Python. Permite criar interfaces de usuário reativas, com componentes, hooks e renderização eficiente.

? Instalação

Via pip (Recomendado)

bash
pip install pyreact-framework

Via pip (GitHub)

bash
pip install git+https://github.com/wanbnn/pyreact.git

Desenvolvimento Local

bash
git clone https://github.com/wanbnn/pyreact.git
cd pyreact
pip install -e .

? Início Rápido

Criar um Novo Projeto

bash

Criar projeto
pyreact-framework create meu-app

Entrar no diretório
cd meu-app

Iniciar servidor de desenvolvimento
pyreact-framework dev

Exemplo de Contador

python
from pyreact import h, render, use_state, Component

def Counter(props):
"""Componente de contador funcional"""
count, set_count = use_state(0)

return h('div', {'className': 'counter'},
h('h1', None, f'Contador: {count}'),
h('button', {'onClick': lambda: set_count(count + 1)}, '+'),
h('button', {'onClick': lambda: set_count(count - 1)}, '-')
)

Renderizar

root = document.getElementById('root')
render(h(Counter, None), root)

? Funcionalidades

Componentes Funcionais

python
def Button(props):
"""Componente de botão reutilizável"""
return h('button', {
'className': 'btn',
'onClick': props.get('onClick')
}, props.get('children'))

Componentes de Classe

python
class Counter(Component):
"""Componente de contador com estado"""

def __init__(self, props):
super().__init__(props)
self.state = {'count': 0}

def render(self):
return h('div', None,
h('h1', None, f'Count: {self.state["count"]}'),
h('button', {
'onClick': lambda: self.set_state({'count': self.state['count'] + 1})
}, 'Increment')
)

Hooks

python
from pyreact import use_state, use_effect, use_ref

def MyComponent(props):

Estado local

```
count, set_count = use_state(0)
```

Efeito colateral

```
use_effect(lambda: print(f'Count: {count}'), [count])
```

Referência

```
input_ref = use_ref(None)

return h('div', None, f'Count: {count}')
```

? Documentação

? Documentação Completa

A documentação completa está disponível no Read the Docs:

? https://pyreact-framework.readthedocs.io/

Conteúdo da Documentação:

- Getting Started
- Installation
- Quick Start
- Tutorial (Todo App)

- Core Concepts
- Components
- Props
- State
- Events
- Lifecycle

- Advanced
- Server-Side Rendering (SSR)
- Routing
- Styling
- Testing

- API Reference
- Element API
- Component API
- Hooks API
- CLI API

CLI Commands

bash

Criar novo projeto

pyreact create <nome>

Iniciar servidor de desenvolvimento

pyreact-framework dev [--port PORT]

Gerar componente
pyreact generate component <nome>

Gerar hook
pyreact generate hook <nome>

Build para produção
pyreact build

API Principal

`h(type, props, *children)`

Cria um elemento virtual (VNode).

python
h('div', {'className': 'container'},
`h('h1', None, 'Título'),`
`h('p', None, 'Parágrafo')`
)

`render(element, container)`

Renderiza um elemento no container.

python
render(h(App, None), document.getElementById('root'))

`create_root(container)`

Cria uma raiz de renderização (API moderna).

python
root = create_root(document.getElementById('root'))
root.render(h(App, None))

? Testes

Executar Testes

bash

Testes unitários
pytest tests/

Testes E2E
pytest tests/e2e/ -v

Com cobertura
pytest tests/ --cov=pyreact

Documentação de Testes

Ver `docs/GUIA_EXECUCAO_E_TESTES.md` para mais detalhes.

? Estrutura do Projeto

```
pyreact/
??? pyreact/            # Código fonte
?   ??? cli/            # Interface de linha de comando
?   ??? core/           # Núcleo do framework
?   ??? dom/            # Operações DOM
?   ??? server/         # Renderização servidor
?   ??? utils/          # Utilitários
??? tests/              # Testes
?   ??? e2e/            # Testes end-to-end
?   ??? unit/           # Testes unitários
??? examples/           # Exemplos
??? docs/               # Documentação e relatórios em PDF
??? pyproject.toml      # Configuração do projeto
??? README.md           # Este arquivo
??? INSTALL.md          # Guia de instalação
??? PUBLISH.md          # Guia de publicação
??? LICENSE             # Licença MIT
```

?? Desenvolvimento

Configurar Ambiente

```
bash
# Clonar repositório
git clone https://github.com/wanbnn/pyreact.git
cd pyreact

# Criar ambiente virtual
python -m venv venv
source venv/bin/activate  # Linux/Mac
venv\Scripts\activate     # Windows

# Instalar dependências
pip install -e .

# Instalar dependências de desenvolvimento
pip install -e ".[dev]"
pip install playwright
playwright install chromium
```

Executar em Desenvolvimento

```
bash
# Instalar em modo editável
pip install -e .
```

Executar testes
pytest tests/ -v

Criar build
python -m build

? Publicação

Build

bash

Limpar builds anteriores
python -c "import shutil; from pathlib import Path; [shutil.rmtree(p, ignore_errors=True) for p in ['build', 'dist', 'pyreact.egg-info']]"

Criar build
python -m build

Publicar no PyPI

bash

Instalar twine
pip install twine

Verificar build
twine check dist/*

Publicar no TestPyPI (teste)
twine upload --repository testpypi dist/*

Publicar no PyPI (oficial)
twine upload dist/*

Ver `PUBLISH.md` para mais detalhes.

? Contribuindo

Contribuições são bem-vindas! Por favor:

1. Fork o projeto
2. Crie uma branch para sua feature (`git checkout -b feature/AmazingFeature`)
3. Commit suas mudanças (`git commit -m 'Add some AmazingFeature'`)
4. Push para a branch (`git push origin feature/AmazingFeature`)
5. Abra um Pull Request

? Licença

Este projeto está licenciado sob a licença MIT - veja o arquivo LICENSE para detalhes.

? Contato

- GitHub Issues: https://github.com/wanbnn/pyreact/issues
- Documentação: https://pyreact.readthedocs.io/
- Email: contato@pyreact.dev

? Agradecimentos

- Inspirado no [React](https://reactjs.org/)
- Construído com ?? pela comunidade Python

Feito com ?? pela comunidade Python

docs/advanced/routing.rst

.. _routing:

Routing

PyReact provides a built-in router for Single Page Applications (SPAs).

Basic Routing

Set up routing in your app:

.. code-block:: python

from pyreact import element, Component
from pyreact.router import Router, Route, Link

class App(Component):
```
def render(self):
return element(Router, {},
element('nav', {},
element(Link, {'to': '/'}, 'Home'),
element(Link, {'to': '/about'}, 'About'),
element(Link, {'to': '/contact'}, 'Contact')
),
element('main', {},
element(Route, {'path': '/', 'component': Home}),
element(Route, {'path': '/about', 'component': About}),
element(Route, {'path': '/contact', 'component': Contact})
)
)
```

Route Parameters

Access route parameters:

.. code-block:: python

from pyreact.router import useParams

class UserProfile(Component):
```
def render(self):
params = useParams()
user_id = params['id']

return element('div', {},
```

```
element('h1', {}, f'User Profile: {user_id}'),
element('p', {}, 'User details here...')
)
```

Route definition

element(Route, {
'path': '/users/:id',
'component': UserProfile
})

Query Parameters

Access query parameters:

.. code-block:: python

from pyreact.router import useLocation

class SearchResults(Component):

```
def render(self):
location = useLocation()
query = location.get('query', '')
page = int(location.get('page', 1))

return element('div', {},
element('h1', {}, f'Search: {query}'),
element('p', {}, f'Page {page}')
)
```

Nested Routes

Create nested routes:

.. code-block:: python

class Dashboard(Component):

```
def render(self):
return element('div', {'class': 'dashboard'},
element('aside', {},
element(Link, {'to': '/dashboard'}, 'Overview'),
element(Link, {'to': '/dashboard/settings'}, 'Settings'),
element(Link, {'to': '/dashboard/profile'}, 'Profile')
),
element('main', {},
element(Route, {'path': '/dashboard', 'exact': True, 'component': DashboardHome}),
element(Route, {'path': '/dashboard/settings', 'component': Settings}),
element(Route, {'path': '/dashboard/profile', 'component': Profile})
)
)
```

Programmatic Navigation

Navigate programmatically:

.. code-block:: python

from pyreact.router import useHistory

class LoginForm(Component):
```
def __init__(self, props):
super().__init__(props)
self.history = useHistory()

def handle_submit(self, event):
event.preventDefault()
```
Perform login
```
success = self.login()

if success:
```
Redirect to dashboard
```
self.history.push('/dashboard')

def render(self):
return element('form', {'onSubmit': self.handle_submit},
```
... form fields
```
)
```

Route Guards

Protect routes with guards:

.. code-block:: python

from pyreact.router import Route, Redirect

class PrivateRoute(Component):
```
def render(self):
is_authenticated = self.props.get('isAuthenticated', False)

if not is_authenticated:
return element(Redirect, {'to': '/login'})

Component = self.props['component']
return element(Component, self.props)
```
# Usage
element(PrivateRoute, {
```
'path': '/dashboard',
'component': Dashboard,
'isAuthenticated': user.is_authenticated
```
})

404 Not Found

Handle 404 pages:

.. code-block:: python

from pyreact.router import Switch, Route

class App(Component):
```
def render(self):
return element(Router, {},
element(Switch, {},
element(Route, {'path': '/', 'component': Home}),
```

```
element(Route, {'path': '/about', 'component': About}),
element(Route, {'component': NotFound})  # 404 fallback
)
)
```

class NotFound(Component):

```
def render(self):
return element('div', {'class': 'not-found'},
element('h1', {}, '404 - Page Not Found'),
element(Link, {'to': '/'}, 'Go Home')
)
```

Best Practices

1. Use meaningful routes - /users/123 instead of /u/123
2. Handle 404s - Always provide a fallback route
3. Protect sensitive routes - Use route guards
4. Lazy load routes - Improve performance with code splitting

Next Steps

- :doc:`/advanced/ssr` - Server-side rendering
- :doc:`/advanced/styling` - Styling options
- :doc:`/api/hooks` - Router hooks

docs/advanced/ssr.rst

.. _ssr:

Server-Side Rendering (SSR)

PyReact supports Server-Side Rendering (SSR) for improved SEO and initial load performance.

What is SSR?

SSR renders your PyReact components on the server and sends the HTML to the client. This provides:

- Better SEO - Search engines can crawl your content
- Faster initial load - Users see content immediately
- Better performance - Reduced client-side JavaScript

Enabling SSR

Enable SSR in your pyproject.toml:

.. code-block:: toml

[tool.pyreact]
ssr = true

SSR Configuration

Configure SSR in your project:

.. code-block:: python
:caption: src/server.py

from pyreact import render_to_string
from my_app import App

def handler(request):

Render app to HTML string

```
html = render_to_string(element(App, {}))

return {
'status': 200,
'headers': {'Content-Type': 'text/html'},
'body': f'''
<!DOCTYPE html>
<html>
<head>
<title>My PyReact App</title>
</head>
<body>
<div id="root">{html}</div>
<script src="/static/client.js"></script>
</body>
</html>
'''
}
```

Hydration

Hydration attaches event listeners to server-rendered HTML:

.. code-block:: python
:caption: src/index.py

```
from pyreact import hydrate
from my_app import App
```

Hydrate the server-rendered HTML

```
hydrate(element(App, {}), root='root')
```

Data Fetching

Fetch data on the server:

.. code-block:: python

```
from pyreact import element, Component

class ProductList(Component):
@staticmethod
async def get_initial_props():
```

Fetch data on server

```
import aiohttp
async with aiohttp.ClientSession() as session:
async with session.get('/api/products') as response:
return await response.json()

def render(self):
products = self.props.get('products', [])
return element('div', {},
*[element('div', {'key': p['id']},
element('h3', {}, p['name']),
```

```
element('p', {}, f"${p['price']}")
) for p in products]
)
```

SSR Best Practices

1. Avoid window/document - These don't exist on the server
2. Check for server environment - Use conditional logic

.. code-block:: python

import os

class MyComponent(Component):
```
def render(self):
```

Check if running on server

```
if os.environ.get('SERVER_SIDE'):
return element('div', {}, 'Server rendered')

return element('div', {},
element('canvas', {'ref': self.canvas_ref})
)
```

3. Handle async data - Use get_initial_props for data fetching

SSR Limitations

- No access to browser APIs (window, document)
- No lifecycle methods during SSR
- State is initialized but not interactive until hydration

Next Steps

- :doc:`/advanced/routing` - Add routing to your app
- :doc:`/advanced/styling` - Learn about styling
- :doc:`/api/component` - Component API reference

docs/advanced/styling.rst

.. _styling:

Styling

PyReact supports multiple styling approaches for your components.

Inline Styles

Apply styles directly to elements:

.. code-block:: python

from pyreact import element

def StyledButton(props):
```
return element('button', {
'style': {
'backgroundColor': '#007bff',
'color': 'white',
'padding': '10px 20px',
```

```
'border': 'none',
'borderRadius': '5px',
'cursor': 'pointer'
}
}, 'Click me')
```

CSS Classes

Use CSS classes for styling:

.. code-block:: python

Component

def Card(props):
```
return element('div', {'class': 'card'},
element('h2', {'class': 'card-title'}, props['title']),
element('p', {'class': 'card-body'}, props['children'])
)
```

.. code-block:: css
:caption: styles/main.css

.card {
```
border: 1px solid #ddd;
border-radius: 8px;
padding: 20px;
margin: 10px 0;
}
```

.card-title {
```
font-size: 24px;
margin-bottom: 10px;
}
```

.card-body {
```
color: #666;
}
```

CSS Modules

Enable CSS Modules in pyproject.toml:

.. code-block:: toml

[tool.pyreact]
css_modules = true

Use CSS Modules in components:

.. code-block:: python

from pyreact import element
from styles import CardStyles # Import CSS module

def Card(props):
```
return element('div', {'class': CardStyles.card},
element('h2', {'class': CardStyles.title}, props['title']),
element('p', {'class': CardStyles.body}, props['children'])
```

)

.. code-block:: css
:caption: styles/CardStyles.module.css

.card {
border: 1px solid #ddd;
border-radius: 8px;
padding: 20px;
}

.title {
font-size: 24px;
color: #333;
}

Styled Components

Create styled components:

.. code-block:: python

from pyreact import styled

Create styled button

Button = styled('button', '''
background-color: #007bff;
color: white;
padding: 10px 20px;
border: none;
border-radius: 5px;
cursor: pointer;

&:hover {
background-color: #0056b3;
}

&:disabled {
background-color: #ccc;
cursor: not-allowed;
}
''')

Usage

element(Button, {'disabled': False}, 'Click me')

Theming

Implement theming:

.. code-block:: python

from pyreact import element, Component, ThemeProvider

class App(Component):
def render(self):
theme = {
'primary': '#007bff',

```
'secondary': '#6c757d',
'success': '#28a745',
'danger': '#dc3545',
'warning': '#ffc107',
'info': '#17a2b8',
'light': '#f8f9fa',
'dark': '#343a40'
}

return element(ThemeProvider, {'theme': theme},
element(ThemedButton, {})
)
```

class ThemedButton(Component):

```
def render(self):
theme = self.use_theme()

return element('button', {
'style': {
'backgroundColor': theme['primary'],
'color': 'white'
}
}, 'Themed Button')
```

Responsive Design

Create responsive layouts:

.. code-block:: python

from pyreact import element

def ResponsiveGrid(props):

```
items = props.get('items', [])

return element('div', {
'style': {
'display': 'grid',
'gridTemplateColumns': 'repeat(auto-fit, minmax(300px, 1fr))',
'gap': '20px',
'padding': '20px'
}
},
*[element('div', {
'style': {
'border': '1px solid #ddd',
'padding': '20px',
'borderRadius': '8px'
}
}, item) for item in items]
)
```

Best Practices

1. Use CSS Modules - Scoped styles prevent conflicts
2. Avoid inline styles - Use classes for better performance
3. Use a theme - Centralize colors and spacing

4. Make it responsive - Design for all screen sizes

Next Steps

- :doc:`/advanced/ssr` - Server-side rendering
- :doc:`/advanced/testing` - Testing components
- :doc:`/api/element` - Element API reference

docs/advanced/testing.rst

.. _testing:

Testing

PyReact provides utilities for testing your components.

Unit Testing

Test components with pytest:

.. code-block:: python
:caption: tests/test_button.py

```
import pytest
from pyreact import element
from pyreact.testing import render, fireEvent
from my_app.components import Button

def test_button_renders():
    """Test that button renders with correct text"""
    result = render(element(Button, {'text': 'Click me'}))
    assert result.text == 'Click me'

def test_button_click():
    """Test that button handles click events"""
    clicked = []

    def handle_click(event):
        clicked.append(True)

    result = render(element(Button, {
        'text': 'Click me',
        'onClick': handle_click
    }))

    fireEvent.click(result)
    assert len(clicked) == 1
```

Testing State

Test component state changes:

.. code-block:: python
:caption: tests/test_counter.py

```
from pyreact import element
from pyreact.testing import render, fireEvent
from my_app.components import Counter

def test_counter_initial_state():
```

```
    """Test counter initial state"""
    result = render(element(Counter, {}))
    assert result.text == 'Count: 0'

def test_counter_increment():
    """Test counter increment"""
    result = render(element(Counter, {}))
```

Click increment button

```
    increment_btn = result.find('button', text='+')
    fireEvent.click(increment_btn)

    assert result.text == 'Count: 1'

def test_counter_decrement():
    """Test counter decrement"""
    result = render(element(Counter, {}))
```

Click decrement button

```
    decrement_btn = result.find('button', text='-')
    fireEvent.click(decrement_btn)

    assert result.text == 'Count: -1'
```

Testing Props

Test component props:

.. code-block:: python
:caption: tests/test_greeting.py

from pyreact import element
from pyreact.testing import render
from my_app.components import Greeting

def test_greeting_with_name():
"""Test greeting with name prop"""
result = render(element(Greeting, {'name': 'PyReact'}))
assert 'PyReact' in result.text

def test_greeting_default():
"""Test greeting with default prop"""
result = render(element(Greeting, {}))
assert 'World' in result.text

Testing Events

Test event handlers:

.. code-block:: python
:caption: tests/test_form.py

from pyreact import element
from pyreact.testing import render, fireEvent
from my_app.components import ContactForm

def test_form_submission():
"""Test form submission"""

```
submitted = []

def handle_submit(event):
event.preventDefault()
submitted.append(event.data)

result = render(element(ContactForm, {
'onSubmit': handle_submit
}))
```

Fill form

```
name_input = result.find('input', {'name': 'name'})
fireEvent.change(name_input, {'value': 'John Doe'})

email_input = result.find('input', {'name': 'email'})
fireEvent.change(email_input, {'value': 'john@example.com'})
```

Submit form

```
form = result.find('form')
fireEvent.submit(form)

assert len(submitted) == 1
assert submitted[0]['name'] == 'John Doe'
assert submitted[0]['email'] == 'john@example.com'
```

Testing Async Components

Test async operations:

.. code-block:: python
:caption: tests/test_data_fetcher.py

import pytest
from pyreact import element
from pyreact.testing import render, waitFor
from my_app.components import DataFetcher

@pytest.mark.asyncio
async def test_data_fetcher():
```
"""Test async data fetching"""
result = render(element(DataFetcher, {}))
```

Initially shows loading

```
assert 'Loading' in result.text
```

Wait for data to load

```
await waitFor(lambda: 'Data loaded' in result.text)
```

Check final state

```
assert 'Data loaded' in result.text
```

Integration Testing

Test component integration:

.. code-block:: python
:caption: tests/test_app_integration.py

from pyreact import element
from pyreact.testing import render, fireEvent
from my_app import App

def test_app_integration():
"""Test full app integration"""
result = render(element(App, {}))

Navigate to about page

about_link = result.find('a', {'href': '/about'})
fireEvent.click(about_link)

Check about page loaded

assert 'About Us' in result.text

Navigate back home

home_link = result.find('a', {'href': '/'})
fireEvent.click(home_link)

Check home page loaded

assert 'Welcome' in result.text

Test Configuration

Configure pytest for PyReact:

.. code-block:: python
:caption: tests/conftest.py

import pytest
from pyreact.testing import TestRenderer

@pytest.fixture
def renderer():
"""Provide test renderer"""
renderer = TestRenderer()
yield renderer
renderer.cleanup()

Best Practices

1. Test behavior, not implementation - Focus on what the component does
2. Use descriptive test names - test_button_displays_error_when_disabled
3. Test edge cases - Empty states, errors, loading states
4. Keep tests isolated - Each test should be independent
5. Use fixtures - Reuse common test setup

Next Steps

- :doc:`/api/component` - Component API reference
- :doc:`/advanced/ssr` - Server-side rendering
- :doc:`/resources/faq` - Frequently asked questions

docs/api/cli.rst

.. _cli:

CLI API

PyReact provides a command-line interface for creating and managing projects.

Installation

The CLI is installed with PyReact:

.. code-block:: bash

pip install pyreact-framework

Commands

pyreact-framework create

Create a new PyReact project.

.. code-block:: bash

pyreact-framework create <project-name> [options]

Arguments:

- project-name - Name of the project (required)

Options:

- --template - Template to use (default: 'default')
- --no-git - Skip git initialization

Examples:

.. code-block:: bash

Create basic project
pyreact-framework create my-app

Create from template
pyreact-framework create my-app --template dashboard

Create without git
pyreact-framework create my-app --no-git

pyreact-framework dev

Start the development server.

.. code-block:: bash

pyreact-framework dev [options]

Options:

- --port - Port number (default: 3000)
- --host - Host address (default: 'localhost')
- --open - Open browser automatically

Examples:

.. code-block:: bash

Start on default port
pyreact-framework dev

Start on custom port
pyreact-framework dev --port 8080

Start and open browser
pyreact-framework dev --open

pyreact-framework build

Build the project for production.

.. code-block:: bash

pyreact-framework build [options]

Options:

- --output - Output directory (default: 'dist')
- --mode - Build mode ('production' or 'development')

Examples:

.. code-block:: bash

Build for production
pyreact-framework build

Build with custom output
pyreact-framework build --output build

pyreact-framework serve

Serve the production build.

.. code-block:: bash

pyreact-framework serve [options]

Options:

- --port - Port number (default: 5000)
- --dir - Directory to serve (default: 'dist')

Examples:

.. code-block:: bash

Serve on default port
pyreact-framework serve

Serve on custom port
pyreact-framework serve --port 8080

pyreact-framework test

Run tests.

.. code-block:: bash

pyreact-framework test [options]

Options:

- --watch - Watch for changes
- --coverage - Generate coverage report
- --pattern - Test file pattern

Examples:

.. code-block:: bash

Run all tests
pyreact-framework test

Run with coverage
pyreact-framework test --coverage

Run specific tests
pyreact-framework test --pattern "test_components"

pyreact-framework --version

Display PyReact version.

.. code-block:: bash

pyreact-framework --version
pyreact-framework -v

pyreact-framework --help

Display help information.

.. code-block:: bash

pyreact-framework --help
pyreact-framework -h

Configuration

Configure your project in pyproject.toml:

.. code-block:: toml

[tool.pyreact]

# Entry point
entry = "src/index.py"

# Output directory
output = "dist"

# Development server
dev_port = 3000
dev_host = "localhost"

# SSR
ssr = true

CSS Modules

css_modules = true

Source maps

source_maps = true

Project Structure

Standard project structure:

.. code-block:: text

my-app/
??? src/
?   ??? index.py   # Entry point
?   ??? components/   # Components
?   ?   ??? Header.py
?   ?   ??? Footer.py
?   ??? styles/   # Styles
?   ?   ??? main.css
?   ??? utils/   # Utilities
??? tests/   # Tests
?   ??? test_components.py
?   ??? test_utils.py
??? docs/   # Documentation
??? pyproject.toml   # Configuration
??? README.md

Environment Variables

PyReact CLI respects these environment variables:

- PYREACT_PORT - Default development server port
- PYREACT_HOST - Default development server host
- PYREACT_OUTPUT - Default build output directory
- NODE_ENV - Environment mode ('development' or 'production')

Best Practices

1. Use version control - Initialize git for your project
2. Configure in pyproject.toml - Centralize configuration
3. Use environment variables - For environment-specific settings
4. Run tests before build - Ensure quality before deployment

Next Steps

- :doc:`/getting-started/quickstart` - Quick start guide
- :doc:`/getting-started/tutorial` - Tutorial
- :doc:`/advanced/testing` - Testing guide

docs/api/component.rst

.. _component:

Component API

Components are the building blocks of PyReact applications.

Component Class

.. py:class:: Component(props)

Base class for all PyReact components.

:param props: Properties passed to the component
:type props: dict

Example:

.. code-block:: python

```
from pyreact import element, Component

class MyComponent(Component):
def __init__(self, props):
super().__init__(props)
self.state = {'count': 0}

def render(self):
return element('div', {}, f'Count: {self.state["count"]}')
```

State Management

.. py:method:: setState(state, callback=None)

Updates the component's state and triggers a re-render.

:param state: New state or function that returns new state
:type state: dict | callable
:param callback: Function to call after state is updated
:type callback: callable, optional

Example:

.. code-block:: python

```
class Counter(Component):
def __init__(self, props):
super().__init__(props)
self.state = {'count': 0}

def increment(self):
```

Update state

```
self.setState({'count': self.state['count'] + 1})

def increment_async(self):
```

Functional update

```
self.setState(lambda state: {'count': state['count'] + 1})

def increment_with_callback(self):
```

With callback

```
self.setState(
{'count': self.state['count'] + 1},
lambda: print(f'Count is now {self.state["count"]}')
)
```

.. py:attribute:: state

The component's state object.

:type: dict

Example:

.. code-block:: python

```
def render(self):
return element('div', {}, f'Count: {self.state["count"]}')
```

.. py:attribute:: props

The component's props object (read-only).

:type: dict

Example:

.. code-block:: python

```
def render(self):
return element('h1', {}, f'Hello, {self.props["name"]}')
```

Lifecycle Methods

.. py:method:: component_did_mount()

Called immediately after the component is mounted.

Use for:
- Fetching data
- Setting up subscriptions
- Adding event listeners

Example:

.. code-block:: python

```
class DataFetcher(Component):
def component_did_mount(self):
self.fetch_data()

async def fetch_data(self):
```

Fetch data from API

```
pass
```

.. py:method:: component_did_update(prev_props, prev_state)

Called after the component updates.

:param prev_props: Previous props
:type prev_props: dict
:param prev_state: Previous state
:type prev_state: dict

Example:

.. code-block:: python

```
class UserViewer(Component):
def component_did_update(self, prev_props, prev_state):
```

```
if prev_props['userId'] != self.props['userId']:
self.fetch_user_data()
```

.. py:method:: component_will_unmount()

Called immediately before the component is unmounted.

Use for:
- Cleaning up subscriptions
- Canceling timers
- Removing event listeners

Example:

.. code-block:: python

```
class Timer(Component):
def component_did_mount(self):
self.timer_id = self.start_timer()

def component_will_unmount(self):
self.stop_timer(self.timer_id)
```

.. py:method:: should_component_update(next_props, next_state)

Called before rendering. Return False to prevent re-render.

:param next_props: Next props
:type next_props: dict
:param next_state: Next state
:type next_state: dict
:returns: Whether the component should update
:rtype: bool

Example:

.. code-block:: python

```
class OptimizedComponent(Component):
def should_component_update(self, next_props, next_state):
```

Only update if data changed

```
return self.props['data'] != next_props['data']
```

.. py:method:: component_did_catch(error, info)

Called when an error is thrown during rendering.

:param error: The error that was thrown
:type error: Exception
:param info: Information about the error
:type info: dict

Example:

.. code-block:: python

```
class ErrorBoundary(Component):
def __init__(self, props):
super().__init__(props)
self.state = {'has_error': False}
```

```
def component_did_catch(self, error, info):
self.setState({'has_error': True})
log_error(error, info)
```

Render Method

.. py:method:: render()

Must be implemented by the component. Returns the element tree.

:returns: The element tree to render
:rtype: Element

Example:

.. code-block:: python

```
class MyComponent(Component):
def render(self):
return element('div', {},
element('h1', {}, 'Title'),
element('p', {}, 'Content')
)
```

Force Update

.. py:method:: force_update(callback=None)

Forces a re-render of the component.

:param callback: Function to call after update
:type callback: callable, optional

Example:

.. code-block:: python

```
class MyComponent(Component):
def some_method(self):
```

Force re-render without changing state

```
self.force_update()
```

Function Components

Function components are simpler:

.. py:function:: FunctionComponent(props)

A function that takes props and returns an element.

:param props: Properties passed to the component
:type props: dict
:returns: An element tree
:rtype: Element

Example:

.. code-block:: python

```
def Greeting(props):
name = props.get('name', 'World')
```

```
return element('h1', {}, f'Hello, {name}!')
```

Best Practices

1. Keep components small - Each component should do one thing
2. Lift state up - Share state by lifting to common ancestor
3. Use should_component_update - Optimize performance
4. Clean up in will_unmount - Always clean up subscriptions

Next Steps

- :doc:`/api/hooks` - Hooks API reference
- :doc:`/concepts/state` - Learn about state
- :doc:`/concepts/lifecycle` - Learn about lifecycle

docs/api/element.rst

.. _element:

Element API

The element() function is the core building block of PyReact. It creates virtual DOM elements.

Function Signature

.. py:function:: element(type, props={}, *children)

Creates a virtual DOM element.

:param type: The type of element. Can be:
- A string (HTML tag name like 'div', 'span', 'h1')
- A component class or function
- A fragment (Fragment)
:type type: str | Component | Fragment

:param props: Properties/attributes for the element
:type props: dict, optional

:param children: Child elements
:type children: element | str | list

:returns: A virtual DOM element
:rtype: Element

Basic Usage

Create HTML elements:

.. code-block:: python

from pyreact import element

# Simple element
element('div')

# Element with props
element('div', {'class': 'container', 'id': 'main'})

Element with children

element('div', {},
element('h1', {}, 'Title'),
element('p', {}, 'Paragraph')
)

HTML Elements

Create standard HTML elements:

.. code-block:: python

# Text content
element('h1', {}, 'Hello World')
element('p', {}, 'This is a paragraph')

# Attributes
element('input', {'type': 'text', 'placeholder': 'Enter name'})
element('a', {'href': 'https://example.com', 'target': '_blank'}, 'Link')

# Styles
element('div', {
'style': {
'backgroundColor': 'red',
'padding': '10px'
}
})

Component Elements

Create component instances:

.. code-block:: python

from pyreact import element
from my_components import Button, Card

# Function component
element(Button, {'text': 'Click me', 'onClick': handle_click})

Class component
element(Card, {'title': 'My Card'},
element('p', {}, 'Card content')
)

Fragments

Group elements without adding extra DOM nodes:

.. code-block:: python

from pyreact import element, Fragment

def ListItem(props):
return element(Fragment, {},
element('dt', {}, props['term']),
element('dd', {}, props['definition'])
)

Children

Pass children in different ways:

.. code-block:: python

As positional arguments

element('div', {},
element('p', {}, 'First'),
element('p', {}, 'Second')
)

As a list

children = [
element('p', {}, 'First'),
element('p', {}, 'Second')
]
element('div', {}, *children)

Mixed

element('ul', {'class': 'list'},
*[element('li', {}, f'Item {i}') for i in range(5)]
)

Special Props

Key

Unique identifier for list items:

.. code-block:: python

element('ul', {},
*[element('li', {'key': item['id']}, item['text'])
for item in items]
)

Ref

Reference to DOM element:

.. code-block:: python

from pyreact import create_ref

class MyComponent(Component):
def __init__(self, props):
super().__init__(props)
self.input_ref = create_ref()

def focus_input(self):
self.input_ref.current.focus()

def render(self):
return element('input', {'ref': self.input_ref})

Style

Inline styles:

.. code-block:: python

element('div', {
'style': {
'color': 'red',
'fontSize': '16px', # camelCase
'marginTop': '10px'
}
})

className

CSS class names:

.. code-block:: python

element('div', {'class': 'container active'})

Multiple classes

element('div', {'class': 'btn btn-primary btn-large'})

dangerouslySetInnerHTML

Insert raw HTML (use with caution):

.. code-block:: python

element('div', {
'dangerouslySetInnerHTML': {'__html': 'Bold'}
})

Event Handlers

Attach event handlers:

.. code-block:: python

element('button', {
'onClick': lambda e: print('Clicked'),
'onMouseEnter': lambda e: print('Mouse entered'),
'onFocus': lambda e: print('Focused')
})

Boolean Attributes

Boolean attributes:

.. code-block:: python

element('input', {
'type': 'checkbox',
'checked': True, # checked
'disabled': False # not disabled
})

Data Attributes

Custom data attributes:

.. code-block:: python

element('div', {
'data-id': '123',
'data-type': 'user',
'data-active': 'true'
})

Best Practices

1. Always use keys for lists - Helps PyReact identify which items changed
2. Use fragments for grouped content - Avoid unnecessary wrapper divs
3. Extract repeated elements - Create components for repeated patterns
4. Keep props simple - Don't pass complex logic in props

Next Steps

- :doc:`/api/component` - Component API reference
- :doc:`/concepts/props` - Learn about props
- :doc:`/concepts/events` - Learn about events

docs/api/hooks.rst

.. _hooks:

Hooks API

Hooks let you use state and other features in function components.

useState

.. py:function:: useState(initial_value)

Adds state to a function component.

:param initial_value: Initial state value
:type initial_value: any
:returns: Tuple of (current_value, setter_function)
:rtype: tuple

Example:

.. code-block:: python

```
from pyreact import element, useState

def Counter(props):
count, set_count = useState(0)

def increment():
set_count(count + 1)

return element('div', {},
element('p', {}, f'Count: {count}'),
element('button', {'onClick': increment}, 'Increment')
)
```

Functional updates:

.. code-block:: python

```
def Counter(props):
```

```
count, set_count = useState(0)

def increment():
```

Use function for derived state

```
set_count(lambda prev: prev + 1)

return element('button', {'onClick': increment}, f'Count: {count}')
```

useEffect

.. py:function:: useEffect(effect, dependencies=None)

Performs side effects in function components.

:param effect: Function to run (can return cleanup function)
:type effect: callable
:param dependencies: Array of values that trigger the effect
:type dependencies: list, optional

Example:

.. code-block:: python

```
from pyreact import element, useState, useEffect

def DataFetcher(props):
data, set_data = useState(None)
loading, set_loading = useState(True)

def fetch_data():
```

Fetch data

```
set_data(fetch_from_api())
set_loading(False)
```

Run once on mount

```
useEffect(fetch_data, [])

if loading:
return element('div', {}, 'Loading...')

return element('div', {}, f'Data: {data}')
```

With cleanup:

.. code-block:: python

```
def Timer(props):
seconds, set_seconds = useState(0)

def setup_timer():
timer_id = setInterval(lambda: set_seconds(s => s + 1), 1000)
```

Cleanup function

```
return lambda: clearInterval(timer_id)

useEffect(setup_timer, [])

return element('div', {}, f'Seconds: {seconds}')
```

With dependencies:

.. code-block:: python

```
def UserProfile(props):
user_id = props['userId']
user, set_user = useState(None)

def fetch_user():
set_user(fetch_user_by_id(user_id))
```

Re-run when userId changes

```
useEffect(fetch_user, [user_id])

return element('div', {}, f'User: {user}')
```

useContext

.. py:function:: useContext(context)

Accesses context value.

:param context: Context object created by createContext
:type context: Context
:returns: Current context value
:rtype: any

Example:

.. code-block:: python

```
from pyreact import element, createContext, useContext

ThemeContext = createContext('light')

def ThemedButton(props):
theme = useContext(ThemeContext)

return element('button', {
'style': {'backgroundColor': theme['primary']}
}, 'Click me')
```

useRef

.. py:function:: useRef(initial_value=None)

Creates a mutable reference that persists across renders.

:param initial_value: Initial ref value
:type initial_value: any, optional
:returns: Ref object with current property
:rtype: Ref

Example:

.. code-block:: python

```
from pyreact import element, useRef

def TextInput(props):
input_ref = useRef()

def focus():
```

```
input_ref.current.focus()

return element('div', {},
element('input', {'ref': input_ref}),
element('button', {'onClick': focus}, 'Focus')
)
```

useMemo

.. py:function:: useMemo(factory, dependencies)

Memoizes expensive computations.

:param factory: Function that computes the value
:type factory: callable
:param dependencies: Values that trigger recomputation
:type dependencies: list
:returns: Memoized value
:rtype: any

Example:

.. code-block:: python

```
from pyreact import element, useMemo

def ExpensiveList(props):
items = props['items']
filter_text = props['filter']
```

Only recompute when items or filter changes

```
filtered_items = useMemo(
lambda: [i for i in items if filter_text in i['name']],
[items, filter_text]
)

return element('ul', {},
*[element('li', {'key': i['id']}, i['name'])
for i in filtered_items]
)
```

useCallback

.. py:function:: useCallback(callback, dependencies)

Returns a memoized callback.

:param callback: Function to memoize
:type callback: callable
:param dependencies: Values that trigger recreation
:type dependencies: list
:returns: Memoized callback
:rtype: callable

Example:

.. code-block:: python

```
from pyreact import element, useCallback
```

```
def Parent(props):
def handle_click(item_id):
print(f'Clicked: {item_id}')
```

Memoize callback

```
memoized_click = useCallback(handle_click, [])

return element('div', {},
*[element(ChildButton, {
'key': item['id'],
'id': item['id'],
'onClick': memoized_click
}) for item in props['items']]
)
```

useReducer

.. py:function:: useReducer(reducer, initial_state, init=None)

Alternative to useState for complex state logic.

:param reducer: Function (state, action) => new_state
:type reducer: callable
:param initial_state: Initial state value
:type initial_state: any
:param init: Lazy initialization function
:type init: callable, optional
:returns: Tuple of (state, dispatch)
:rtype: tuple

Example:

.. code-block:: python

```
from pyreact import element, useReducer

def reducer(state, action):
if action['type'] == 'increment':
return {'count': state['count'] + 1}
elif action['type'] == 'decrement':
return {'count': state['count'] - 1}
elif action['type'] == 'reset':
return {'count': action['payload']}
return state

def Counter(props):
state, dispatch = useReducer(reducer, {'count': 0})

return element('div', {},
element('p', {}, f'Count: {state["count"]}'),
element('button', {
'onClick': lambda: dispatch({'type': 'increment'})
}, '+'),
element('button', {
'onClick': lambda: dispatch({'type': 'decrement'})
}, '-'),
element('button', {
```

```
'onClick': lambda: dispatch({'type': 'reset', 'payload': 0})
}, 'Reset')
)
```

Custom Hooks

Create custom hooks:

.. code-block:: python

from pyreact import useState, useEffect

def useLocalStorage(key, initial_value):
"""Custom hook for localStorage"""
stored = localStorage.getItem(key)
value, set_value = useState(stored or initial_value)

def save_to_storage():
localStorage.setItem(key, value)

useEffect(save_to_storage, [value])

return value, set_value

Usage

def App(props):
name, set_name = useLocalStorage('name', 'Guest')

return element('input', {
'value': name,
'onChange': lambda e: set_name(e.target.value)
})

Best Practices

1. Only call hooks at the top level - Not inside loops or conditions
2. Only call hooks from function components - Not from regular functions
3. Use the dependency array - Specify all dependencies
4. Name custom hooks with "use" - Convention for custom hooks

Next Steps

- :doc:`/api/component` - Component API reference
- :doc:`/concepts/state` - Learn about state
- :doc:`/advanced/testing` - Testing hooks

docs/concepts/components.rst

.. _components:

Components

Components are the building blocks of PyReact applications. They are reusable pieces of UI that can manage their own state and props.

Component Types

PyReact supports two types of components:

1. Function Components - Simple components defined as functions

2. Class Components - Complex components with state and lifecycle methods

Function Components

Function components are simple and concise:

.. code-block:: python

from pyreact import element

def Greeting(props):
```
name = props.get('name', 'World')
return element('h1', {}, f'Hello, {name}!')
```

Usage

element(Greeting, {'name': 'PyReact'})

Class Components

Class components offer more features:

.. code-block:: python

from pyreact import element, Component

class Counter(Component):
```
def __init__(self, props):
super().__init__(props)
self.state = {'count': 0}

def increment(self, event):
self.setState({'count': self.state['count'] + 1})

def render(self):
return element('div', {},
element('p', {}, f'Count: {self.state["count"]}'),
element('button', {'onClick': self.increment}, 'Increment')
)
```

Component Lifecycle

Class components have lifecycle methods:

.. code-block:: python

class MyComponent(Component):
```
def component_did_mount(self):
"""Called after the component is mounted"""
print('Component mounted')

def component_did_update(self, prev_props, prev_state):
"""Called after the component updates"""
print('Component updated')

def component_will_unmount(self):
"""Called before the component is unmounted"""
print('Component will unmount')

def render(self):
return element('div', {}, 'My Component')
```

Component Composition

Components can be composed together:

.. code-block:: python

```
def Header(props):
return element('header', {},
element('h1', {}, props['title'])
)

def Footer(props):
return element('footer', {},
element('p', {}, props['text'])
)

def App(props):
return element('div', {},
element(Header, {'title': 'My App'}),
element('main', {}, 'Content goes here'),
element(Footer, {'text': '© 2026'})
)
```

Best Practices

1. Keep components small - Each component should do one thing well
2. Use meaningful names - Component names should be descriptive
3. Reuse components - Create reusable components for common patterns
4. Lift state up - Share state between components by lifting it to their common ancestor

Next Steps

- :doc:`/concepts/props` - Learn about props
- :doc:`/concepts/state` - Learn about state management
- :doc:`/api/component` - Component API reference

docs/concepts/events.rst

.. _events:

Events

PyReact uses synthetic events to handle user interactions. Events are similar to DOM events but work consistently across all browsers.

Event Handling

Handle events by passing functions to element props:

.. code-block:: python

```
from pyreact import element, Component

class Button(Component):
def handle_click(self, event):
print('Button clicked!')

def render(self):
return element('button', {
```

```
'onClick': self.handle_click
}, 'Click me')
```

Event Types

PyReact supports all standard DOM events:

Mouse Events

.. code-block:: python

element('div', {
'onClick': lambda e: print('Clicked'),
'onDoubleClick': lambda e: print('Double clicked'),
'onMouseEnter': lambda e: print('Mouse entered'),
'onMouseLeave': lambda e: print('Mouse left'),
'onMouseMove': lambda e: print(f'Position: {e.clientX}, {e.clientY}')
})

Keyboard Events

.. code-block:: python

element('input', {
'onKeyDown': lambda e: print(f'Key down: {e.key}'),
'onKeyUp': lambda e: print(f'Key up: {e.key}'),
'onKeyPress': lambda e: print(f'Key pressed: {e.key}')
})

Form Events

.. code-block:: python

element('form', {
'onSubmit': lambda e: e.preventDefault()
},
element('input', {
'onChange': lambda e: print(f'Value: {e.target.value}')
}),
element('button', {'type': 'submit'}, 'Submit')
)

Focus Events

.. code-block:: python

element('input', {
'onFocus': lambda e: print('Focused'),
'onBlur': lambda e: print('Blurred')
})

Event Object

The event object contains useful properties:

.. code-block:: python

def handle_click(self, event):

Prevent default behavior

```
event.preventDefault()
```

Stop event propagation

```
event.stopPropagation()
```

Access event properties

```
print(f'Target: {event.target}')
print(f'Current target: {event.currentTarget}')
print(f'Type: {event.type}')
print(f'TimeStamp: {event.timeStamp}')
```

Passing Arguments

Pass arguments to event handlers:

.. code-block:: python

class TodoList(Component):

```
def delete_todo(self, todo_id, event):
event.preventDefault()
print(f'Deleting todo: {todo_id}')

def render(self):
todos = self.props['todos']
return element('ul', {},
*[element('li', {'key': todo['id']},
element('span', {}, todo['text']),
element('button', {
'onClick': lambda e, id=todo['id']: self.delete_todo(id, e)
}, 'Delete')
) for todo in todos]
)
```

Conditional Event Handlers

Conditionally attach event handlers:

.. code-block:: python

class Button(Component):

```
def render(self):
disabled = self.props.get('disabled', False)

return element('button', {
'disabled': disabled,
'onClick': None if disabled else self.handle_click
}, 'Click me')
```

Event Pooling

PyReact uses event pooling for performance. Access event properties asynchronously:

.. code-block:: python

def handle_click(self, event):

Persist the event to use it asynchronously

```
event.persist()
```

Now you can use it in async code

```
import asyncio
asyncio.create_task(self.async_operation(event))
```

async def async_operation(self, event):

```
await asyncio.sleep(1)
print(f'Event type: {event.type}')
```

Best Practices

1. Use descriptive names - handle_submit instead of submit
2. Prevent default when needed - Use event.preventDefault() for forms
3. Clean up event listeners - Remove listeners in component_will_unmount
4. Avoid inline functions - Define methods instead of lambdas for complex logic

Next Steps

- :doc:`/concepts/lifecycle` - Learn about lifecycle methods
- :doc:`/api/element` - Element API reference
- :doc:`/advanced/routing` - Add routing to your app

docs/concepts/lifecycle.rst

.. _lifecycle:

Lifecycle Methods

Lifecycle methods allow you to run code at specific times in a component's life.

Mounting

Methods called when a component is being added to the DOM:

component_did_mount()

Called immediately after the component is mounted. Use this for:

- Fetching data
- Setting up subscriptions
- Adding event listeners

.. code-block:: python

class DataFetcher(Component):

```
def __init__(self, props):
super().__init__(props)
self.state = {'data': None, 'loading': True}

def component_did_mount(self):
```

Fetch data when component mounts

```
self.fetch_data()

async def fetch_data(self):
```

Simulated data fetch

```
import asyncio
await asyncio.sleep(1)
self.setState({
'data': {'message': 'Hello from API'},
```

```
'loading': False
})

def render(self):
if self.state['loading']:
return element('div', {}, 'Loading...')

return element('div', {}, self.state['data']['message'])
```

Updating

Methods called when a component is being re-rendered:

component_did_update(prev_props, prev_state)

Called after the component updates. Use this for:

- Responding to prop changes
- Network requests based on new data
- DOM updates

.. code-block:: python

class UserProfile(Component):
```
def component_did_update(self, prev_props, prev_state):
```

Fetch new data when user ID changes

```
if prev_props.get('userId') != self.props.get('userId'):
self.fetch_user_data()

def fetch_user_data(self):
user_id = self.props['userId']
print(f'Fetching data for user {user_id}')

def render(self):
return element('div', {}, f'User: {self.props["userId"]}')
```

should_component_update(nextProps, nextState)

Called before rendering. Return False to skip rendering:

.. code-block:: python

class ExpensiveComponent(Component):
```
def should_component_update(self, next_props, next_state):
```

Only update if data actually changed

```
return self.props['data'] != next_props['data']

def render(self):
```

Expensive rendering operation

```
return element('div', {}, 'Expensive content')
```

Unmounting

Methods called when a component is being removed from the DOM:

component_will_unmount()

Called immediately before the component is unmounted. Use this for:

- Cleaning up subscriptions

- Canceling network requests
- Removing event listeners

.. code-block:: python

class Timer(Component):
```
def __init__(self, props):
super().__init__(props)
self.state = {'seconds': 0}
self.timer_id = None

def component_did_mount(self):
self.timer_id = self.start_timer()

def component_will_unmount(self):
```
Clean up timer
```
if self.timer_id:
self.stop_timer(self.timer_id)

def start_timer(self):
import threading
def tick():
self.setState({'seconds': self.state['seconds'] + 1})

timer = threading.Timer(1.0, tick)
timer.start()
return timer

def stop_timer(self, timer_id):
timer_id.cancel()

def render(self):
return element('div', {}, f'Seconds: {self.state["seconds"]}')
```

Error Handling

Methods for handling errors during rendering:

component_did_catch(error, info)

Called when an error is thrown during rendering:

.. code-block:: python

class ErrorBoundary(Component):
```
def __init__(self, props):
super().__init__(props)
self.state = {'has_error': False, 'error': None}

def component_did_catch(self, error, info):
```
Log error to service
```
print(f'Error: {error}')
print(f'Info: {info}')

self.setState({
'has_error': True,
'error': error
})
```

```
def render(self):
if self.state['has_error']:
return element('div', {'class': 'error'},
element('h2', {}, 'Something went wrong'),
element('p', {}, str(self.state['error']))
)

return self.props.get('children', [])
```

Lifecycle Diagram

.. code-block:: text

Mounting:
constructor -> render -> component_did_mount

Updating:
new props/state -> should_component_update -> render -> component_did_update

Unmounting:
component_will_unmount

Best Practices

1. Clean up in will_unmount - Always clean up subscriptions and timers
2. Avoid side effects in constructor - Use component_did_mount instead
3. Use should_component_update - Optimize performance for expensive renders
4. Handle errors - Use error boundaries for better error handling

Next Steps

- :doc:`/api/component` - Component API reference
- :doc:`/advanced/testing` - Test lifecycle methods
- :doc:`/concepts/state` - Learn about state management

docs/concepts/props.rst

.. _props:

Props

Props (short for "properties") are how data flows from parent to child components. They are read-only and help make components reusable.

Basic Usage

Pass props to a component:

.. code-block:: python

from pyreact import element

def Greeting(props):
```
name = props.get('name', 'World')
return element('h1', {}, f'Hello, {name}!')
```

# Pass props
element(Greeting, {'name': 'PyReact'})

Props are Read-Only

Components must never modify their own props:

.. code-block:: python
:caption: ? Wrong

```
def Greeting(props):
props['name'] = 'Modified'  # Never do this!
return element('h1', {}, f'Hello, {props["name"]}')
```

.. code-block:: python
:caption: ? Correct

```
def Greeting(props):
name = props.get('name', 'World')
return element('h1', {}, f'Hello, {name}')
```

Default Props

Provide default values for props:

.. code-block:: python

```
def Button(props):
text = props.get('text', 'Click me')
variant = props.get('variant', 'primary')
disabled = props.get('disabled', False)

return element('button', {
'class': f'btn btn-{variant}',
'disabled': disabled
}, text)
```

Usage

```
element(Button, {})  # Uses defaults
element(Button, {'text': 'Submit', 'variant': 'success'})
```

Children Prop

Pass children to components:

.. code-block:: python

```
def Card(props):
title = props.get('title', '')
children = props.get('children', [])

return element('div', {'class': 'card'},
element('h2', {'class': 'card-title'}, title),
element('div', {'class': 'card-body'}, *children)
)
```

Usage

```
element(Card, {'title': 'Welcome'},
element('p', {}, 'This is the card content'),
element('button', {}, 'Click me')
)
```

Prop Types

Props can be any Python type:

.. code-block:: python

```
def UserCard(props):
user = props['user']  # dict
on_click = props['onClick']  # function
is_active = props.get('active', False)  # bool
tags = props.get('tags', [])  # list

return element('div', {'class': 'user-card'},
element('h3', {}, user['name']),
element('p', {}, f"Active: {is_active}"),
element('div', {},
*[element('span', {'class': 'tag'}, tag) for tag in tags]
),
element('button', {'onClick': on_click}, 'View Profile')
)
```

Passing Functions as Props

Pass callback functions to child components:

.. code-block:: python

```
class Parent(Component):
def __init__(self, props):
super().__init__(props)
self.state = {'count': 0}

def handle_increment(self, amount):
self.setState({'count': self.state['count'] + amount})

def render(self):
return element('div', {},
element('p', {}, f'Count: {self.state["count"]}'),
element(ChildButton, {
'onClick': lambda e: self.handle_increment(1),
'label': 'Add 1'
}),
element(ChildButton, {
'onClick': lambda e: self.handle_increment(10),
'label': 'Add 10'
})
)
```

Best Practices

1. Use descriptive prop names - is_active instead of active
2. Provide defaults - Use props.get() with default values
3. Document props - Add docstrings explaining expected props
4. Validate props - Check required props exist

Next Steps

- :doc:`/concepts/state` - Learn about state
- :doc:`/concepts/events` - Learn about event handling
- :doc:`/api/component` - Component API reference

docs/concepts/state.rst

.. _state:

State

State represents data that can change over time. Unlike props, state is managed by the component itself and can be updated.

Initializing State

Initialize state in the constructor:

.. code-block:: python

from pyreact import element, Component

class Counter(Component):
```
def __init__(self, props):
super().__init__(props)
self.state = {'count': 0}
```

Updating State

Use setState() to update state:

.. code-block:: python

class Counter(Component):
```
def __init__(self, props):
super().__init__(props)
self.state = {'count': 0}

def increment(self, event):
```
? Correct: Use setState
```
self.setState({'count': self.state['count'] + 1})

def bad_increment(self, event):
```
? Wrong: Never modify state directly
```
self.state['count'] += 1  # Don't do this!
```

State Updates are Merged

setState() merges the new state with the existing state:

.. code-block:: python

class Form(Component):
```
def __init__(self, props):
super().__init__(props)
self.state = {
'name': '',
'email': '',
'age': 0
}

def update_name(self, event):
```
Only updates 'name', preserves 'email' and 'age'
```
self.setState({'name': event.target.value})
```

```
def update_email(self, event):
self.setState({'email': event.target.value})
```

Functional Updates

When new state depends on previous state, use a function:

.. code-block:: python

class Counter(Component):

```
def __init__(self, props):
super().__init__(props)
self.state = {'count': 0}

def increment(self, event):
```

Use function for derived state

```
self.setState(lambda state: {'count': state['count'] + 1})

def increment_three_times(self, event):
```

Multiple updates

```
self.setState(lambda state: {'count': state['count'] + 1})
self.setState(lambda state: {'count': state['count'] + 1})
self.setState(lambda state: {'count': state['count'] + 1})
```

State is Local

State is local to the component:

.. code-block:: python

class App(Component):

```
def render(self):
return element('div', {},
element(Counter, {}),  # Has its own state
element(Counter, {}),  # Has its own state
element(Counter, {})   # Has its own state
)
```

Lifting State Up

Share state between components by lifting it to their common ancestor:

.. code-block:: python

class App(Component):

```
def __init__(self, props):
super().__init__(props)
self.state = {'temperature': 0}

def handle_change(self, temp):
self.setState({'temperature': temp})

def render(self):
return element('div', {},
element(TemperatureInput, {
'temperature': self.state['temperature'],
'onChange': self.handle_change
}),
```

```
element(BoilingVerdict, {
'celsius': self.state['temperature']
})
)
```

State vs Props

.. list-table::
:widths: 50 50
:header-rows: 1

* - Props
  - State
* - Passed from parent
  - Managed by component
* - Read-only
  - Can be updated
* - Used for configuration
  - Used for dynamic data
* - External to component
  - Internal to component

Best Practices

1. Minimize state - Keep state as simple as possible
2. Lift state up - Share state by lifting to common ancestor
3. Don't duplicate state - Compute derived values instead
4. Use setState - Never modify state directly

Next Steps

- :doc:`/concepts/events` - Learn about event handling
- :doc:`/api/hooks` - Explore built-in hooks
- :doc:`/concepts/lifecycle` - Learn about lifecycle methods

docs/getting-started/installation.rst

.. _installation:

Installation

PyReact Framework requires Python 3.10 or higher.

Stable Release

Install the latest stable version from PyPI:

.. code-block:: bash

pip install pyreact-framework

Development Version

Install the latest development version from GitHub:

.. code-block:: bash

pip install git+https://github.com/wanbnn/pyreact.git

Requirements

PyReact Framework automatically installs the following dependencies:

- watchdog>=3.0.0 - File system monitoring for hot reload
- jinja2>=3.1.0 - Template rendering
- pyyaml>=6.0 - YAML configuration parsing
- click>=8.1.0 - CLI framework (alternative to argparse)

Verification

Verify your installation:

.. code-block:: bash

pyreact-framework --version

You should see output similar to:

.. code-block:: text

PyReact Framework v1.0.5

Next Steps

- :doc:`/getting-started/quickstart` - Create your first PyReact app
- :doc:`/getting-started/tutorial` - Follow the step-by-step tutorial

docs/getting-started/quickstart.rst

.. _quickstart:

Quick Start

Create your first PyReact application in less than 5 minutes.

Create a New Project

Use the CLI to create a new project:

.. code-block:: bash

pyreact-framework create my-app
cd my-app

This creates a project structure:

.. code-block:: text

my-app/
??? src/
?   ??? index.py   # Entry point
?   ??? components/   # Reusable components
?   ??? styles/   # CSS files
??? pyproject.toml   # Project configuration
??? README.md

Start Development Server

Start the development server with hot reload:

.. code-block:: bash

pyreact-framework dev

The application will be available at http://localhost:3000.

Your First Component

Create a simple counter component:

.. code-block:: python
:caption: src/components/Counter.py

from pyreact import element, Component

class Counter(Component):
```
def __init__(self, props):
super().__init__(props)
self.state = {'count': 0}

def increment(self, event):
self.setState({'count': self.state['count'] + 1})

def decrement(self, event):
self.setState({'count': self.state['count'] - 1})

def render(self):
return element('div', {'class': 'counter'},
element('h2', {}, f'Count: {self.state["count"]}'),
element('button', {'onClick': self.decrement}, '-'),
element('button', {'onClick': self.increment}, '+')
)
```

Use the Component

Import and use your component:

.. code-block:: python
:caption: src/index.py

from pyreact import element, render
from components.Counter import Counter

def App():
```
return element('div', {'class': 'app'},
element('h1', {}, 'My First PyReact App'),
element(Counter, {})
)
```

render(App(), root='root')

Build for Production

Build your application for production:

.. code-block:: bash

pyreact-framework build

The build output will be in the dist/ directory.

Next Steps

- :doc:`/getting-started/tutorial` - Learn PyReact in depth
- :doc:`/concepts/components` - Understand components

- :doc:`/api/hooks` - Explore built-in hooks

docs/getting-started/tutorial.rst

.. _tutorial:

Tutorial: Building a Todo App

In this tutorial, you'll build a complete Todo application with PyReact Framework.

What You'll Build

A fully functional Todo app with:

- Add new todos
- Mark todos as complete
- Delete todos
- Filter todos (all/active/completed)
- Persistent storage (localStorage)

Prerequisites

- Python 3.10 or higher
- PyReact Framework installed
- Basic Python knowledge

Step 1: Create the Project

.. code-block:: bash

pyreact-framework create todo-app
cd todo-app
pyreact-framework dev

Step 2: Create TodoItem Component

Create src/components/TodoItem.py:

.. code-block:: python

from pyreact import element, Component

class TodoItem(Component):
```
def render(self):
todo = self.props['todo']
on_toggle = self.props['onToggle']
on_delete = self.props['onDelete']

return element('li', {
'class': f'todo-item {"completed" if todo["completed"] else ""}'
},
element('input', {
'type': 'checkbox',
'checked': todo['completed'],
'onChange': lambda e: on_toggle(todo['id'])
}),
element('span', {'class': 'todo-text'}, todo['text']),
element('button', {
'class': 'delete-btn',
```

```
'onClick': lambda e: on_delete(todo['id'])
}, '×')
)
```

Step 3: Create TodoList Component

Create src/components/TodoList.py:

.. code-block:: python

```
from pyreact import element, Component
from .TodoItem import TodoItem

class TodoList(Component):
def render(self):
todos = self.props['todos']
filter_type = self.props.get('filter', 'all')

filtered_todos = todos
if filter_type == 'active':
filtered_todos = [t for t in todos if not t['completed']]
elif filter_type == 'completed':
filtered_todos = [t for t in todos if t['completed']]

return element('ul', {'class': 'todo-list'},
*[element(TodoItem, {
'todo': todo,
'onToggle': self.props['onToggle'],
'onDelete': self.props['onDelete']
}) for todo in filtered_todos]
)
```

Step 4: Create App Component

Update src/index.py:

.. code-block:: python

```
from pyreact import element, Component, render
from components.TodoList import TodoList

class App(Component):
def __init__(self, props):
super().__init__(props)
self.state = {
'todos': [],
'input': '',
'filter': 'all'
}

def add_todo(self, event):
if self.state['input'].strip():
new_todo = {
'id': len(self.state['todos']) + 1,
'text': self.state['input'],
'completed': False
}
self.setState({
```

```
'todos': self.state['todos'] + [new_todo],
'input': ''
})

def toggle_todo(self, todo_id):
todos = self.state['todos'][:]
for todo in todos:
if todo['id'] == todo_id:
todo['completed'] = not todo['completed']
self.setState({'todos': todos})

def delete_todo(self, todo_id):
todos = [t for t in self.state['todos'] if t['id'] != todo_id]
self.setState({'todos': todos})

def set_filter(self, filter_type):
self.setState({'filter': filter_type})

def render(self):
return element('div', {'class': 'app'},
element('h1', {}, 'Todo App'),
element('div', {'class': 'add-todo'},
element('input', {
'type': 'text',
'value': self.state['input'],
'onInput': lambda e: self.setState({'input': e.target.value}),
'placeholder': 'Add a todo...'
}),
element('button', {'onClick': self.add_todo}, 'Add')
),
element(TodoList, {
'todos': self.state['todos'],
'filter': self.state['filter'],
'onToggle': self.toggle_todo,
'onDelete': self.delete_todo
}),
element('div', {'class': 'filters'},
element('button', {
'class': f"filter-btn {'' if self.state['filter'] == 'all' else ''}",
'onClick': lambda e: self.set_filter('all')
}, 'All'),
element('button', {
'class': f"filter-btn {'' if self.state['filter'] == 'active' else ''}",
'onClick': lambda e: self.set_filter('active')
}, 'Active'),
element('button', {
'class': f"filter-btn {'' if self.state['filter'] == 'completed' else ''}",
'onClick': lambda e: self.set_filter('completed')
}, 'Completed')
)
)
```

render(element(App, {}), root='root')

Step 5: Add Styles

Create src/styles/main.css:

.. code-block:: css

```
.app {
max-width: 600px;
margin: 0 auto;
padding: 20px;
}

.add-todo {
display: flex;
gap: 10px;
margin-bottom: 20px;
}

.add-todo input {
flex: 1;
padding: 10px;
font-size: 16px;
}

.todo-list {
list-style: none;
padding: 0;
}

.todo-item {
display: flex;
align-items: center;
gap: 10px;
padding: 10px;
border-bottom: 1px solid #eee;
}

.todo-item.completed .todo-text {
text-decoration: line-through;
opacity: 0.5;
}

.delete-btn {
background: #ff4444;
color: white;
border: none;
border-radius: 50%;
width: 30px;
height: 30px;
cursor: pointer;
}

.filters {
display: flex;
gap: 10px;
margin-top: 20px;
}
```

.filter-btn.active {
```
background: #007bff;
color: white;
```
}

Step 6: Run and Test

Your app is ready! Open http://localhost:3000 and test:

1. Add a new todo
2. Mark it as complete
3. Filter by active/completed
4. Delete a todo

What's Next?

- :doc:`/concepts/state` - Learn more about state management
- :doc:`/advanced/routing` - Add routing to your app
- :doc:`/advanced/testing` - Write tests for your components

docs/index.rst

.. PyReact Framework documentation master file

Welcome to PyReact Framework's Documentation!

PyReact Framework é um framework web declarativo inspirado no React, construído nativamente para Python.

:maxdepth: 2
:caption: Getting Started

getting-started/installation
getting-started/quickstart
getting-started/tutorial

:maxdepth: 2
:caption: Core Concepts

concepts/components
concepts/props
concepts/state
concepts/events
concepts/lifecycle

:maxdepth: 2
:caption: Advanced

advanced/ssr
advanced/routing
advanced/styling
advanced/testing

:maxdepth: 2
:caption: API Reference

api/element

api/component
api/hooks
api/cli

:maxdepth: 1
:caption: Resources

resources/faq
resources/changelog
resources/contributing

Indices and tables

* genindex
* modindex
* search

docs/resources/changelog.rst

.. _changelog:

Changelog

All notable changes to PyReact Framework are documented here.

Version 1.0.5 (2026-07-24)

Added

- Portable end-to-end tests for scaffold, CLI, build and browser interactions
- A complete boilerplate project demonstrating PyReact in a realistic task board
- Reproducible PDF documentation and release/test reports
- Both pyreact and pyreact-framework CLI entry points
- Functional production build output and dev --no-open for CI

Fixed

- Component state is committed when no renderer scheduler is attached
- Hook state is isolated per component and persists across renders
- Function components re-render after hook state updates
- Child reconciliation updates text and removes stale nodes correctly
- Re-rendered event handlers replace previous listeners instead of accumulating
- SSR hydration markers are emitted on the root node only
- Testing utilities now mount the real DOM, dispatch events and support cleanup/rerender
- Generated projects and custom hooks contain executable tests and valid hook syntax

Validation

- 101 framework tests passing
- 5 boilerplate tests passing, including Chromium E2E
- Source distribution and wheel validated before publication

Version 1.0.1 (2026-03-28)

Fixed

- Updated README with correct GitHub username (wanbnn)
- Fixed PyPI badges to use correct package name (pyreact-framework)

- Updated project URLs in pyproject.toml

Version 1.0.0 (2026-03-28)

Added

- Initial stable release
- Core component system with Element API
- Class components with state management
- Function components with hooks
- Lifecycle methods (component_did_mount, component_did_update, component_will_unmount)
- Event handling system
- Built-in router for SPAs
- CSS Modules support
- Server-side rendering (SSR)
- Hot module replacement for development
- CLI tools (create, dev, build, serve, test)
- Comprehensive test suite
- Full documentation

Features

Components

- Element creation with element() function
- Class components extending Component
- Function components
- Props and state management
- Component composition

Hooks

- useState - State management
- useEffect - Side effects
- useContext - Context API
- useRef - Mutable references
- useMemo - Memoization
- useCallback - Callback memoization
- useReducer - Complex state logic

Routing

- Declarative routing
- Route parameters
- Query parameters
- Nested routes
- Route guards
- Programmatic navigation

Styling

- Inline styles
- CSS classes
- CSS Modules
- Styled components
- Theming support

Developer Experience

- Hot reload
- Error boundaries
- Source maps
- TypeScript-like type hints
- Comprehensive error messages

Documentation

- Getting Started guide
- Tutorial (Todo app)
- API reference
- Advanced topics
- FAQ

Roadmap

Version 1.1.0 (Planned)

- TypeScript support
- More hooks (useTransition, useDeferredValue)
- Improved SSR performance
- Better error messages
- More examples

Version 1.2.0 (Planned)

- Animation library
- Form validation
- Internationalization (i18n)
- State management library
- Component library

Version 2.0.0 (Future)

- Concurrent rendering
- Suspense for data fetching
- Server components
- Streaming SSR
- New architecture

Contributing

We welcome contributions! See :doc:`/resources/contributing` for guidelines.

Versioning

PyReact follows Semantic Versioning <https://semver.org/>:

- MAJOR version for incompatible API changes
- MINOR version for new features (backwards compatible)
- PATCH version for bug fixes (backwards compatible)

Previous Versions

- 1.0.0 - Initial stable release (2026-03-28)

Next Steps

- :doc:`/getting-started/installation` - Install PyReact
- :doc:`/getting-started/quickstart` - Quick start guide

- :doc:`/resources/contributing` - Contribute to PyReact

docs/resources/contributing.rst

.. _contributing:

Contributing Guide

Thank you for your interest in contributing to PyReact Framework!

Table of Contents

:local:
:depth: 2

Code of Conduct

Please read and follow our Code of Conduct
<https://github.com/wanbnn/pyreact/blob/main/CODE_OF_CONDUCT.md>.

How to Contribute

Reporting Bugs

1. Check if the bug has already been reported in Issues <https://github.com/wanbnn/pyreact/issues>
2. If not, create a new issue with:
- Clear title
- Steps to reproduce
- Expected behavior
- Actual behavior
- Environment details

Suggesting Features

1. Check if the feature has been suggested
2. Create a new issue with:
- Clear title
- Use case description
- Proposed solution (optional)

Pull Requests

Development Setup

1. Fork the repository
2. Clone your fork:

.. code-block:: bash

```
git clone https://github.com/wanbnn/pyreact.git
cd pyreact
```

3. Create a virtual environment:

.. code-block:: bash

```
python -m venv venv
source venv/bin/activate  # Linux/Mac
venv\Scripts\activate     # Windows
```

4. Install dependencies:

.. code-block:: bash

```
pip install -e ".[dev]"
```

5. Create a branch:

.. code-block:: bash

```
git checkout -b feature/my-feature
```

Coding Standards

Follow these guidelines:

Python Style

- Follow PEP 8 <https://pep8.org/>
- Use type hints
- Write docstrings

Code Example:

.. code-block:: python

```
def calculate_total(items: list[dict]) -> float:
"""
Calculate the total price of items.

Args:
items: List of item dictionaries with 'price' key

Returns:
Total price as float

Raises:
ValueError: If items list is empty
"""
if not items:
raise ValueError("Items list cannot be empty")

return sum(item['price'] for item in items)
```

Testing

Write tests for your code:

.. code-block:: python
:caption: tests/test_utils.py

```
import pytest
from pyreact.utils import calculate_total

def test_calculate_total():
items = [{'price': 10}, {'price': 20}]
assert calculate_total(items) == 30

def test_calculate_total_empty():
with pytest.raises(ValueError):
calculate_total([])
```

Run tests:

.. code-block:: bash

pytest tests/

Documentation

Update documentation:

1. Update docstrings
2. Update API reference if needed
3. Add examples
4. Update changelog

Commit Messages

Follow conventional commits:

.. code-block:: text

type(scope): subject

body (optional)

footer (optional)

Types:

- feat - New feature
- fix - Bug fix
- docs - Documentation
- style - Formatting
- refactor - Code refactoring
- test - Adding tests
- chore - Maintenance

Examples:

.. code-block:: text

feat(hooks): add useTransition hook

Implement useTransition hook for managing transition state.

Closes #123

.. code-block:: text

fix(router): handle nested routes correctly

Fix issue where nested routes were not rendering properly.

Submitting PRs

1. Push your branch:

.. code-block:: bash

```
git push origin feature/my-feature
```

2. Create a Pull Request on GitHub
3. Fill in the PR template

4. Wait for review

Review Process

All PRs require:

1. At least one approval
2. All tests passing
3. No merge conflicts
4. Documentation updated

Reviewers will check:

- Code quality
- Test coverage
- Documentation
- Performance impact
- Breaking changes

Release Process

1. Update version in pyproject.toml
2. Update CHANGELOG.md
3. Create a release PR
4. Merge to main
5. Tag the release:

.. code-block:: bash

```
git tag v1.0.0
git push origin v1.0.0
```

6. GitHub Actions will publish to PyPI

Getting Help

- GitHub Discussions: https://github.com/wanbnn/pyreact/discussions
- Discord: https://discord.gg/pyreact
- Email: wanbnn@users.noreply.github.com

License

By contributing, you agree that your contributions will be licensed under the MIT License.

Thank You!

Your contributions make PyReact better for everyone. Thank you for your time and effort!

Next Steps

- :doc:`/getting-started/installation` - Install PyReact
- :doc:`/getting-started/tutorial` - Build a Todo app
- :doc:`/api/hooks` - Explore hooks

docs/resources/faq.rst

.. _faq:

Frequently Asked Questions

General

What is PyReact Framework?

PyReact Framework is a declarative web framework for Python, inspired by React. It allows you to build web applications using Python instead of JavaScript.

Why use Python for frontend?

- Python is easier to learn and read
- Large ecosystem of libraries
- Same language for frontend and backend
- Great for data science and ML integration

Is PyReact production-ready?

Yes! PyReact is stable and used in production applications. However, it's still evolving, so check the changelog for updates.

Getting Started

How do I install PyReact?

.. code-block:: bash

pip install pyreact-framework

What Python version do I need?

Python 3.10 or higher is required.

How do I create a new project?

.. code-block:: bash

pyreact-framework create my-app
cd my-app
pyreact-framework dev

Components

What's the difference between function and class components?

Function components are simpler and recommended for most cases. Class components are useful when you need state or lifecycle methods.

How do I pass data to components?

Use props:

.. code-block:: python

element(MyComponent, {'name': 'John', 'age': 30})

How do I handle events?

Pass event handlers as props:

.. code-block:: python

element('button', {'onClick': lambda e: print('Clicked')}, 'Click me')

State Management

How do I update state?

Use setState() in class components:

.. code-block:: python

self.setState({'count': self.state['count'] + 1})

Or useState hook in function components:

.. code-block:: python

count, set_count = useState(0)
set_count(count + 1)

Why isn't my state updating?

Make sure you're using setState() or the setter function, not modifying state directly:

.. code-block:: python
:caption: ? Wrong

self.state['count'] += 1 # Don't do this!

.. code-block:: python
:caption: ? Correct

self.setState({'count': self.state['count'] + 1})

How do I share state between components?

Lift state up to the common ancestor component and pass it down as props.

Styling

How do I add CSS?

Import CSS files in your components:

.. code-block:: python

from styles import main_css

Or use inline styles:

.. code-block:: python

element('div', {'style': {'color': 'red'}})

Does PyReact support CSS Modules?

Yes! Enable in pyproject.toml:

.. code-block:: toml

[tool.pyreact]
css_modules = true

Routing

How do I add routing?

Use the built-in router:

.. code-block:: python

from pyreact.router import Router, Route

element(Router, {},
element(Route, {'path': '/', 'component': Home}),
element(Route, {'path': '/about', 'component': About})
)

How do I get route parameters?

Use useParams:

.. code-block:: python

from pyreact.router import useParams

params = useParams()
user_id = params['id']

Testing

How do I test components?

Use pytest with PyReact's testing utilities:

.. code-block:: python

from pyreact.testing import render, fireEvent

result = render(element(Button, {}))
assert result.text == 'Click me'

How do I test async code?

Use asyncio and waitFor:

.. code-block:: python

from pyreact.testing import waitFor

await waitFor(lambda: 'Loaded' in result.text)

Deployment

How do I build for production?

.. code-block:: bash

pyreact-framework build

Where do I deploy PyReact apps?

PyReact apps can be deployed to:

- Vercel
- Netlify
- AWS
- Google Cloud
- Any static hosting
- Python web servers (with SSR)

How do I enable SSR?

Enable in pyproject.toml:

.. code-block:: toml

[tool.pyreact]
ssr = true

Troubleshooting

My component isn't rendering

Check:
1. Is the component exported correctly?
2. Are there any errors in the console?
3. Is the parent component rendering?

Hot reload isn't working

Make sure:
1. Development server is running
2. Files are being saved
3. No syntax errors exist

Build is failing

Check:
1. All imports are correct
2. No circular dependencies
3. All dependencies are installed

Getting Help

- Documentation: https://pyreact-framework.readthedocs.io/
- GitHub Issues: https://github.com/wanbnn/pyreact/issues
- Discussions: https://github.com/wanbnn/pyreact/discussions

Next Steps

- :doc:`/getting-started/tutorial` - Follow the tutorial
- :doc:`/api/hooks` - Explore hooks
- :doc:`/resources/changelog` - See what's new